

Objetivo

Construir formularios en Angular que permitan la interacción con los datos del backend, así como comprender el uso de algunas directivas de Angular para mejorar la experiencia del usuario.

Conceptos Básicos

- **Lenguaje HTML:** Estructura básica de páginas web.
- **Hojas de estilo CSS:** Personalización visual de interfaces.
- **TypeScript:** Superset de JavaScript que añade tipado estático y características avanzadas.
- **Formularios Reactivos en Angular 21:** Uso de `ReactiveFormsModule` integrado armónicamente con la reactividad moderna (Signals) para crear formularios dinámicos y validaciones robustas.
- **Control de Flujo Moderno:** Manipulación del DOM mediante bloques nativos integrados como `@if` y `@for`.
- **signal() / computed()** : Primitivas de reactividad de Angular 21 para gestionar estado de UI de forma síncrona y predecible.
- **toSignal()** : Función de `@angular/core/rxjs-interop` que convierte un `Observable` (como `FormGroup.statusChanges`) en un `Signal` de solo lectura.
- **takeUntilDestroyed()** : Operador RxJS que cancela automáticamente una subscripción cuando el componente es destruido, evitando fugas de memoria.
- **DestroyRef** : Token de inyección que expone el ciclo de vida de destrucción de un componente; usado por `takeUntilDestroyed`.

Contextualización Teórica

¿Qué son los Formularios Reactivos en Angular 21?

Los formularios reactivos son una técnica para construir formularios basados en un modelo de programación reactiva, el cual es el estándar por excelencia en Angular 21. A diferencia de los formularios basados en plantillas (Template-driven forms), los formularios reactivos ofrecen:

- **Mayor control:** Se gestionan desde el componente TypeScript.
- **Validaciones personalizadas:** Facilitan la implementación de reglas de negocio complejas.
- **Escalabilidad:** Son ideales para aplicaciones grandes y complejas.

Estructura de un Formulario Reactivo

Un formulario reactivo se compone de:

1. **FormGroup:** Representa un formulario completo.
2. **FormControl:** Representa un campo individual dentro del formulario.
3. **Validators:** Define reglas de validación para los campos.
4. **FormBuilder:** Simplifica la creación de formularios complejos.

Ventajas de los Formularios Reactivos

- Mantienen la lógica del formulario en el componente, separándola de la vista.
- Permiten la gestión de estados y validaciones de manera eficiente.
- Facilitan la integración con APIs RESTful.

Signals y Formularios Reactivos — ¿cuándo usar cada uno?

Una pregunta muy natural en Angular 21 es: *¿por qué no usamos `signal()` directamente para los campos del formulario?*

La respuesta está en la **separación de responsabilidades**:

Responsabilidad	Herramienta correcta	Motivo
Valor, validación y estado de un campo (<code>touched</code> , <code>dirty</code> , <code>errors</code>)	<code>FormControl</code> / <code>FormGroup</code>	Ya son reactivos internamente; gestionan toda la lógica de validación

Responsabilidad	Herramienta correcta	Motivo
Estado de UI derivado (loading, mensaje de resultado, error de servidor)	signal()	Estado simple y síncrono; ideal para signals
Estado computado a partir del formulario (¿puedo enviar?)	computed() + toSignal()	Puente que convierte el Observable del form en un Signal
Cancelar subscripciones al destruir el componente	takeUntilDestroyed()	Evita fugas de memoria sin código adicional

El patrón Angular 21 recomendado para producción es, por tanto, **Reactive Forms + Signals de UI**, no uno u otro:

```
// ✅ FormGroup gestiona el campo y sus validaciones
loginForm = inject(FormBuilder).group({
  username: ['', [Validators.required, Validators.email]],
  password: ['', [Validators.required, Validators.minLength(6)]]
});

// ✅ signal() gestiona estado de UI
isLoading = signal(false);
result    = signal('');

// ✅ toSignal() convierte el Observable de estado del form en un Signal
formStatus = toSignal(this.loginForm.statusChanges, { initialValue: 'INVALID' as const });

// ✅ computed() deriva lógica: solo se puede enviar si el form es válido y no se está cargar
canSubmit = computed(() => this.formStatus() === 'VALID' && !this.isLoading());
```

Nota — Signal Forms (experimental): Angular 21 incluye un paquete `@angular/forms/signals` con una API completamente basada en signals para formularios. A la fecha (2026) está marcada como **experimental** y su API aún puede cambiar. Se mencionará al final de la guía para que la conozcas, pero el patrón enseñado aquí es el estándar estable recomendado para producción.

Precauciones y Recomendaciones

- Asegúrate de tener instalado Node.js en su última versión.
- Verifica que Angular CLI esté correctamente instalado (`ng version`).

- Usa herramientas como [Angular Language Service](#) en Visual Studio Code para mejorar la productividad.

Artefactos Requeridos

- Visual Studio Code (última versión).
- [WebStorm](#) (Como alternativa a Visual Studio).
- Controlador de versiones GIT.
- Node.js y Angular CLI instalados.

Evaluación o Resultado Esperado

Se espera que el estudiante:

1. Comprenda cómo crear formularios reactivos en Angular.
2. Implemente validaciones básicas sobre campos de un formulario de login.
3. Use el bloque `@if` para mostrar mensajes de error al usuario.
4. Integre el formulario con el endpoint real `POST /api/auth/login` del backend.
5. Almacene el JWT recibido y lo envíe en peticiones autenticadas mediante un `HttpInterceptor`.

Procedimiento

1. Configuración Inicial

1. Asegúrate de que el proyecto Angular de la Guía 14 esté funcionando:

```
npm start
```

2. Verifica que el backend Spring Boot esté corriendo en `http://localhost:8080`.

El backend expone dos endpoints que se usarán en esta guía:

- `POST /api/auth/login` → recibe `{username, password}`, devuelve un JWT.
- `GET /api/solicitudes`, `POST /api/solicitudes` → requieren el header `Authorization: Bearer <token>`.

2. Formulario de Login

1. El componente `Login` ya fue generado en la Guía 14. Abre el archivo `src/app/componentes/login/login.ts`.
2. En el archivo `login.html`, crea la estructura HTML del formulario de ingreso. Los campos deben coincidir exactamente con el DTO `LoginRequest` del backend (`username` y `password`):

```
<form>
  <div>
    <label>Correo electrónico:</label>
    <input type="email" />
  </div>

  <div>
    <label>Contraseña:</label>
    <input type="password" />
  </div>

  <button type="submit">Ingresar</button>
  <div>{{result()}}</div>
</form>
```

La variable `result` mostrará el mensaje de éxito o error de la operación.

3. Importa `ReactiveFormsModule` en el componente `Login` (`login.ts`):

```
import { Component, signal } from '@angular/core';
import { FormBuilder, FormGroup, ReactiveFormsModule, Validators } from '@angular/forms';

@Component({
  selector: 'app-login',
  imports: [ReactiveFormsModule],
  templateUrl: './login.html',
  styleUrls: ['./login.css']
})
export class Login {
  result = signal('');
}
```

4. Inyecta el `FormBuilder` con `inject()` y construye el modelo del formulario en `login.ts`.

Los campos `username` y `password` deben coincidir **exactamente** con el record `LoginRequest(String username, String password)` del backend.

En Angular 21 **no se usa constructor()** para inicializar el formulario. Se declara directamente como campo de clase, lo que mejora la legibilidad y es consistente con la inicialización de signals.

```
import { Component, inject, signal } from '@angular/core';
import { FormBuilder, ReactiveFormsModule, Validators } from '@angular/forms';

@Component({
  selector: 'app-login',
  imports: [ReactiveFormsModule],
  templateUrl: './login.html',
  styleUrls: ['./login.css']
})
export class Login {
  // FormBuilder inyectado inline – sin constructor
  loginForm = inject(FormBuilder).group({
    username: ['', [Validators.required, Validators.email]],
    password: ['', [Validators.required, Validators.minLength(6)]]
  });

  result = signal('');

  onSubmit(): void {
    console.log('Credenciales:', this.loginForm.value);
    this.result.set('Intentando ingresar...');
  }
}
```

5. Enlaza el modelo con la vista en `login.html` . El botón usa `canSubmit()` en lugar de `loginForm.invalid` para reflejar también el estado de carga:

```

<form [formGroup]="loginForm" (ngSubmit)="onSubmit()">
  <div>
    <label>Correo electrónico:</label>
    <input type="email" formControlName="username" />
  </div>

  <div>
    <label>Contraseña:</label>
    <input type="password" formControlName="password" />
  </div>

  <button type="submit" [disabled]="!canSubmit()">
    @if (isLoading()) { Ingresando... } @else { Ingresar }
  </button>
  <div>{{result()}}</div>
</form>

```

6. Ejecuta la aplicación y verifica que el formulario se renderiza correctamente:

```
npm start
```

Abre la consola del navegador (F12) y observa el objeto que se imprime al hacer clic en **Ingresar**.

3. Validación de campos

1. Agrega el bloque `@if` en `login.html` para mostrar errores sobre el campo `username` :

```

<div>
  <label>Correo electrónico:</label>
  <input type="email" formControlName="username" />
  @if (loginForm.get('username')?.invalid && loginForm.get('username')?.touched) {
    <div class="error">Ingresa un correo electrónico válido.</div>
  }
</div>

```

El bloque `@if` está integrado en Angular moderno; no requiere importar módulos adicionales.

2. Agrega el mensaje de error para el campo `password` .

3. Para estilizar los errores, agrega en `login.css` :

```
.error {  
  color: red;  
  font-size: 0.85rem;  
}
```

4. Ejecuta la aplicación, deja un campo en blanco y presiona **Tab** para verificar que los mensajes aparecen.

```
npm start
```

4. Mostrando errores

Esta sección ya está integrada en el paso anterior. Úsala para añadir validaciones a cualquier campo adicional que incorpores en el futuro.

5. Integración con el Backend Real

5.1 Modelos TypeScript — el espejo de los DTOs Java

Nota sobre estructura de carpetas: La guía 14 sugería la carpeta `dto/` para interfaces de datos. A partir de esta guía se usa `modelos/` — nombre más semántico en Angular. Si ya creaste `dto/`, puedes renombrarla o crear `modelos/` junto a ella sin problema.

1. Crea el archivo de modelos de autenticación. Este archivo debe reflejar exactamente los `record` del backend:

```
ng generate interface modelos/auth
```

Edita `src/app/modelos/auth.ts`:


```
// Espejo de: record LoginRequest(String username, String password)
export interface LoginRequest {
  username: string;
  password: string;
}

// Espejo de: record TokenResponse(String token, String type,
//                                     Instant expireAt, Collection<String> roles)
export interface TokenResponse {
  token: string;
  type: string;
  expireAt: string;    // Instant llega como cadena ISO-8601
  roles: string[];
}
```

2. Crea el archivo de modelos de solicitudes:

```
ng generate interface modelos/solicitudes
```

Edita `src/app/modelos/solicitudes.ts` :

```

// Enums del dominio (deben coincidir con los enum Java del backend)
export type TipoSolicitud =
  'REGISTRO_ASIGNATURA' | 'CANCELACION_ASIGNATURA' | 'HOMOLOGACION' | 'OTROS';

export type EstadoSolicitud =
  'REGISTRADA' | 'CLASIFICADA' | 'EN_ATENCION' | 'ATENDIDA' | 'CERRADA' | 'CANCELADA';

// Espejo de: enum Prioridad { BAJA, MEDIA, ALTA }
export type Prioridad = 'BAJA' | 'MEDIA' | 'ALTA';

// Espejo de: record SolicitudResumenResponse(...)
// Nota: 'fechaCreacion' es Instant en Java; Jackson lo serializa como string ISO-8601.
export interface SolicitudResumen {
  id: string;
  codigo: string;
  estado: EstadoSolicitud;
  tipo: TipoSolicitud;
  prioridad: Prioridad;           // presente en SolicitudResumenResponse del backend
  fechaCreacion: string;         // Instant → "2026-04-30T00:00:00Z"
  nombrePropietario: string;
}

// Espejo de: record CrearSolicitudRequest(TipoSolicitud tipo,
//                                     String descripcion, String solicitanteId)
export interface CrearSolicitudRequest {
  tipo: TipoSolicitud;
  descripcion: string;
  solicitanteId: string;
}

```

5.2 Servicio de Autenticación

3. Habilita HttpClient en src/app/app.config.ts :

```
import { ApplicationConfig } from '@angular/core';
import { provideRouter } from '@angular/router';
import { provideHttpClient, withInterceptors } from '@angular/common/http';
import { routes } from './app.routes';
import { authInterceptor } from './interceptores/auth.interceptor';

export const appConfig: ApplicationConfig = {
  providers: [
    provideRouter(routes),
    provideHttpClient(withInterceptors([authInterceptor]))
  ]
};
```

4. Crea el servicio de autenticación:

```
ng generate service servicios/auth --skip-tests
```

Edita src/app/servicios/auth.ts :

```
import { Injectable, inject } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { LoginRequest, TokenResponse } from '../modelos/auth';

@Injectable({ providedIn: 'root' })
export class AuthService {
  private http = inject(HttpClient);
  private url = 'http://localhost:8080/api/auth';

  // Endpoint real del backend: POST /api/auth/login
  login(credenciales: LoginRequest) {
    return this.http.post<TokenResponse>(`${this.url}/login`, credenciales);
  }
}
```

5.3 Conectando el Login con el backend

5. Actualiza login.ts para inyectar el servicio y manejar la respuesta JWT con el patrón moderno Angular 21 (signals de UI + takeUntilDestroyed):

```

import { Component, DestroyRef, computed, inject, signal } from '@angular/core';
import { takeUntilDestroyed, toSignal } from '@angular/core/rxjs-interop';
import { FormBuilder, ReactiveFormsModule, Validators } from '@angular/forms';
import { Router } from '@angular/router';
import { AuthService } from '../../servicios/auth';
import { LoginRequest } from '../../modelos/auth';

@Component({
  selector: 'app-login',
  imports: [ReactiveFormsModule],
  templateUrl: './login.html',
  styleUrls: ['./login.css']
})
export class Login {
  private authService = inject(AuthService);
  private router = inject(Router);
  private destroyRef = inject(DestroyRef);

  // — Modelo del formulario (sin constructor)
  loginForm = inject(FormBuilder).group({
    username: ['', [Validators.required, Validators.email]],
    password: ['', [Validators.required, Validators.minLength(6)]]
  });

  // — Estado de UI
  isLoading = signal(false);
  result = signal('');

  // — Puente Observable → Signal: estado del formulario
  formStatus = toSignal(this.loginForm.statusChanges, { initialValue: 'INVALID' as const });

  // — Estado derivado: ¿se puede enviar?
  canSubmit = computed(() => this.formStatus() === 'VALID' && !this.isLoading());

  onSubmit(): void {
    if (!this.canSubmit()) return;
    this.isLoading.set(true);
    const credenciales = this.loginForm.value as LoginRequest;

    this.authService.login(credenciales)
      .pipe(takeUntilDestroyed(this.destroyRef)) // — cleanup automático al destruir compon
      .subscribe({
        next: (response) => {
          // Guardamos el JWT para usarlo en peticiones posteriores
          localStorage.setItem('token', response.token);
        }
      });
  }
}

```

```

        localStorage.setItem('roles', JSON.stringify(response.roles));
        this.result.set('Sesión iniciada correctamente');
        this.router.navigate(['/']);
    },
    error: () => {
        this.isLoading.set(false);
        this.result.set('Credenciales inválidas. Verifica tu correo y contraseña.');
```

¿Por qué `takeUntilDestroyed` ? Si el usuario navega a otra página antes de que el servidor responda, el componente `Login` es destruido. Sin `takeUntilDestroyed`, la subscripción seguiría activa e intentaría ejecutar `this.router.navigate` sobre un componente ya destruido. Este operador cancela la subscripción automáticamente.

5.4 HttpInterceptor — autorización automática

6. Crea el interceptor que añade el token a cada petición:

```
ng generate interceptor interceptores/auth --skip-tests
```

Edita `src/app/interceptores/auth.interceptor.ts`:

```

import { inject } from '@angular/core';
import { HttpInterceptorFn } from '@angular/common/http';
import { AuthService } from '../servicios/auth';

export const authInterceptor: HttpInterceptorFn = (req, next) => {
    // Usamos AuthService en lugar de leer localStorage directamente
    const token = inject(AuthService).getToken();
    if (token) {
        const petitionAutenticada = req.clone({
            setHeaders: { Authorization: `Bearer ${token}` }
        });
        return next(petitionAutenticada);
    }
    return next(req); // Sin token: la petición sale sin modificar (ej: login)
};
```

5.5 Servicio de Solicitudes

7. Crea el servicio para consumir las solicitudes (ya con el interceptor activo, no hay que gestionar el token manualmente):

```
ng generate service servicios/solicitudes --skip-tests
```

Edita `src/app/servicios/solicitudes.ts` :

```
import { Injectable, inject } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { SolicitudResumen, CrearSolicitudRequest } from '../modelos/solicitudes';

@Injectable({ providedIn: 'root' })
export class SolicitudesService {
  private http = inject(HttpClient);
  private url = 'http://localhost:8080/api/solicitudes';

  listar() {
    return this.http.get<SolicitudResumen[]>(this.url);
  }

  crear(solicitud: CrearSolicitudRequest) {
    return this.http.post<SolicitudResumen>(this.url, solicitud);
  }
}
```

Nota REST: POST `/api/solicitudes` devuelve 201 Created (no 200 OK). HttpClient acepta ambos códigos 2xx transparentemente, pero es importante conocerlo para Postman y para las pruebas.

5.6 Configurar CORS en el Backend

8. Para que el backend acepte solicitudes desde Angular (`localhost:4200`), modifica el archivo `SecurityConfig.java` del proyecto `solicitudes` :

```
// co.edu.uniquindio.ingesis.solicitudes.infrastructure.security.SecurityConfig
@Bean
public SecurityFilterChain defaultSecurityFilterChain(HttpSecurity http) throws Exception {
    return http
        .csrf(AbstractHttpConfigurer::disable)
        .cors(cors -> cors.configurationSource(corsConfigurationSource())) // <- Añadir
        .headers(headers -> headers.frameOptions(f -> f.sameOrigin()))
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/api/auth/login", "/public/**", "/error",
                "/api-docs/**", "/swagger-ui/**", "/swagger-ui.html",
                "/h2-console/**").permitAll()
            .requestMatchers("/api/solicitudes/**").authenticated()
            .anyRequest().authenticated()
        )
        .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .oauth2ResourceServer(oauth2 -> oauth2.jwt(jwt -> jwt.decoder(jwtDecoder)))
        .build();
}

@Bean
public CorsConfigurationSource corsConfigurationSource() {
    CorsConfiguration config = new CorsConfiguration();
    config.setAllowedOrigins(List.of("http://localhost:4200"));
    config.setAllowedMethods(List.of("GET", "POST", "PUT", "PATCH", "DELETE", "OPTIONS"));
    config.setAllowedHeaders(List.of("*"));
    UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
    source.registerCorsConfiguration("/**", config);
    return source;
}
```

9. Reinicia el backend y verifica en la consola del navegador que ya no aparece el error CORS al hacer login.

5.7 Formulario de Registro de Solicitudes

Con el interceptor JWT activo y el servicio listo, construimos el formulario para **crear una solicitud** conectado al endpoint `POST /api/solicitudes`.

1. Genera el componente:

```
ng generate component componentes/nueva-solicitud --skip-tests
```

2. Añade la ruta en `src/app/app.routes.ts`:

```
import { NuevaSolicitud } from './componentes/nueva-solicitud/nueva-solicitud';

export const routes: Routes = [
  { path: '', component: Inicio },
  { path: 'login', component: Login },
  { path: 'nueva-solicitud', component: NuevaSolicitud },
  { path: '**', pathMatch: 'full', redirectTo: '/' },
];
```

3. Implementa el componente `nueva-solicitud.ts` . Los campos deben coincidir exactamente con el record `CrearSolicitudRequest(TipoSolicitud tipo, String descripcion, String solicitanteId)` del backend:


```

import { Component, DestroyRef, inject, signal } from '@angular/core';
import { takeUntilDestroyed } from '@angular/core/rxjs-interop';
import { FormBuilder, ReactiveFormsModule, Validators } from '@angular/forms';
import { Router } from '@angular/router';
import { SolicitudesService } from '../../../servicios/solicitudes';
import { CrearSolicitudRequest, TipoSolicitud } from '../../../modelos/solicitudes';

@Component({
  selector: 'app-nueva-solicitud',
  imports: [ReactiveFormsModule],
  templateUrl: './nueva-solicitud.html',
  styleUrls: ['./nueva-solicitud.css']
})
export class NuevaSolicitud {
  private solicitudesService = inject(SolicitudesService);
  private router = inject(Router);
  private destroyRef = inject(DestroyRef);

  result = signal('');
  isLoading = signal(false);

  // Opciones válidas del enum TipoSolicitud del backend
  tipos: TipoSolicitud[] = [
    'REGISTRO_ASIGNATURA',
    'CANCELACION_ASIGNATURA',
    'HOMOLOGACION',
    'OTROS'
  ];

  solicitudForm = inject(FormBuilder).group({
    tipo: ['', Validators.required],
    descripcion: ['', [Validators.required, Validators.minLength(10), Validators.maxLength(50)
    // Nota: solicitanteId es el UUID del usuario autenticado.
    // En guías posteriores aprenderemos a extraerlo del JWT.
    solicitanteId: ['', Validators.required]
  });

  onSubmit(): void {
    if (this.solicitudForm.invalid) return;
    const solicitud = this.solicitudForm.value as CrearSolicitudRequest;
    this.isLoading.set(true);

    this.solicitudesService.crear(solicitud)
      .pipe(takeUntilDestroyed(this.destroyRef))
      .subscribe({

```

```
next: (response) => {
  this.result.set(`Solicitud creada: ${response.codigo}`);
  this.solicitudForm.reset();
  this.isLoading.set(false);
},
error: () => {
  this.result.set('Error al crear la solicitud. Verifica los datos e intenta de nuevo');
  this.isLoading.set(false);
}
});
}
```

4. Crea la vista `nueva-solicitud.html` enlazada al modelo:

```

<form [formGroup]="solicitudForm" (ngSubmit)="onSubmit()">
  <div>
    <label>Tipo de solicitud:</label>
    <select formControlName="tipo">
      @for (tipo of tipos; track tipo) {
        <option [value]="tipo">{{ tipo }}</option>
      }
    </select>
    @if (solicitudForm.get('tipo')?.invalid && solicitudForm.get('tipo')?.touched) {
      <div class="error">Selecciona un tipo de solicitud.</div>
    }
  </div>

  <div>
    <label>Descripción:</label>
    <textarea formControlName="descripcion" rows="4"></textarea>
    @if (solicitudForm.get('descripcion')?.invalid && solicitudForm.get('descripcion')?.touch
      <div class="error">La descripción debe tener entre 10 y 500 caracteres.</div>
    }
  </div>

  <div>
    <label>ID del solicitante:</label>
    <input type="text" formControlName="solicitanteId" />
    @if (solicitudForm.get('solicitanteId')?.invalid && solicitudForm.get('solicitanteId')?.t
      <div class="error">El ID del solicitante es obligatorio.</div>
    }
  </div>

  <button type="submit" [disabled]="solicitudForm.invalid">Registrar Solicitud</button>
  <div>{{ result() }}</div>
</form>

```

Nota sobre solicitanteId : Actualmente se ingresa como texto. En la guía de **Guards e Interceptores** aprenderemos a decodificar el JWT almacenado en `localStorage` para obtener el identificador del usuario autenticado automáticamente.

5. Añade el enlace al menú en `src/app/app.html` :

```

<li><a routerLink="/nueva-solicitud">Nueva Solicitud</a></li>

```

6. Ejecuta la aplicación, inicia sesión con `admin@solicitudes.com` / `admin123` para obtener el token y luego navega a `/nueva-solicitud` para probar el formulario.

6. Avance del proyecto final

1. Cree los DTOs necesarios para implementar los servicios de su proyecto final.
2. Cree los servicios para su proyecto final.

Para la Próxima Clase

1. Investigue como manejar archivos en sus formularios.
2. Investiga sobre **validaciones personalizadas avanzadas** en Angular.
 - Ejemplo: Validar campos únicos en tiempo real.
 - Referencia: [Custom Validators in Angular](#)
3. Explora cómo usar **interceptores HTTP** para manejar errores globales y autenticación en Angular.
 - Referencia: [HTTP Interceptors in Angular](#)
4. Familiarízate con **ngrx/store** para la gestión de estado en aplicaciones Angular.
 - Referencia: [NgRx Official Documentation](#)

Referencias Bibliográficas

- [Angular Reactive Forms Documentation](#)
- [Angular toSignal — RxJS Interop](#)
- [Angular takeUntilDestroyed](#)
- [Angular Signal Forms \(experimental\)](#)
- [Medium: Angular Reactive Forms](#)
- [W3Schools: Angular Forms](#)